

Detector de Ataques SYN Flood

Baseado em Taxa de Conexões Semiabertas (ISH)

Documentação Técnica do Projeto

Jader Sauzem Volpato
João Vitor Lemos Reis

Biblioteca **Scapy** (Python) - Licença GPLv2
Repositório: <https://github.com/DennyIsBack/detector-syn-flood>

2026

1. Introducao e Software Escolhido

Este documento apresenta a documentacao tecnica do **Detector de Ataques SYN Flood**, desenvolvido em **Python** com a biblioteca **Scapy**. O Scapy e uma biblioteca de codigo aberto (licenca GPLv2) voltada a manipulacao, captura, analise e geracao de pacotes de rede, permitindo construir pacotes, capturar trafego em interfaces de rede e interpretar diversos protocolos de comunicacao.

Neste trabalho o Scapy e utilizado para implementar um detector de ataques do tipo SYN Flood, por meio da analise de pacotes TCP/IP e da verificacao das flags SYN e ACK, monitorando o fluxo de conexoes TCP e identificando o padrao caracteristico do ataque: o envio excessivo de pacotes SYN sem a conclusao do *three-way handshake*.

1.1. Justificativa da escolha do software

O Scapy foi escolhido por reunir as seguintes vantagens:

- Codigo-fonte aberto e ampla documentacao com comunidade ativa;
- Suporte a captura de trafego em tempo real;
- Facilidade de manipulacao e interpretacao de cabecalhos de protocolos;
- Integracao nativa com Python, facilitando funcionalidades personalizadas;
- Flexibilidade para implementar mecanismos de monitoramento e deteccao.

2. Descricao da Funcionalidade

Detector de Ataques SYN Flood baseado em taxa de conexoes semiabertas. A deteccao se baseia na assimetria entre o numero de pacotes SYN recebidos e de pacotes SYN-ACK enviados/retransmitidos pelo servidor. Como as conexoes semiabertas provocam retransmissoes de SYN-ACK (governadas pelo RTO com *backoff* exponencial), um aumento na quantidade de SYN-ACK em relacao aos SYN indica a presenca do ataque.

O sistema acompanha as requisicoes SYN recebidas, os SYN-ACK enviados em resposta e a conclusao (ou nao) do handshake, mantendo a contagem dentro de uma janela de tempo. A partir dessas contagens e calculado um indice (ISH) que dispara alertas quando o trafego foge do padrao normal.

3. Fundamentacao: o ataque SYN Flood

Em uma conexao legitima ocorre o *three-way handshake*: o cliente envia SYN, o servidor responde com SYN-ACK e reserva recursos, e o cliente conclui com ACK, estabelecendo a conexao.

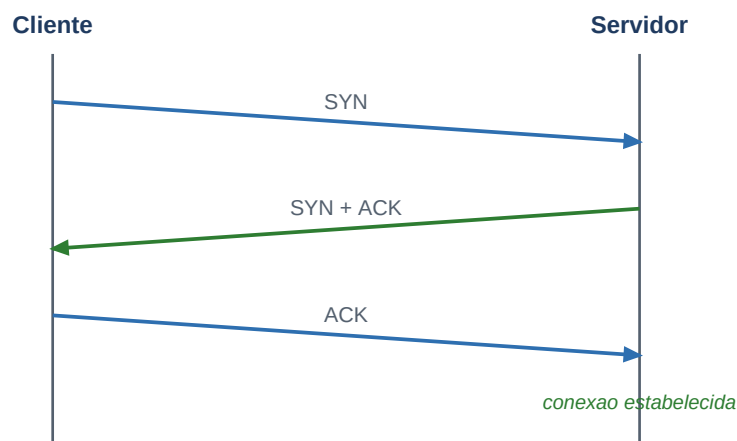


Figura 1 - Three-way handshake TCP em uma conexão legítima.

Durante um ataque SYN Flood, o invasor envia uma grande quantidade de pacotes SYN com endereços falsificados (*spoofing*) ou inexistentes. O servidor responde com SYN-ACK e reserva recursos para cada solicitação, mas o ACK final nunca chega. As conexões permanecem em estado **semiaberto** (half-open) até expirarem por timeout, consumindo recursos do servidor.

Enquanto aguarda, o servidor retransmite o SYN-ACK. Tomando como base o RFC 1122 (RTO inicial de 3 segundos) e o RFC 2988, a cada retransmissão o tempo de espera é dobrado (**back off the timer**: $RTO = RTO \times 2$). Assim, para cada SYN do atacante, múltiplos SYN-ACK são emitidos até o fim do timeout.

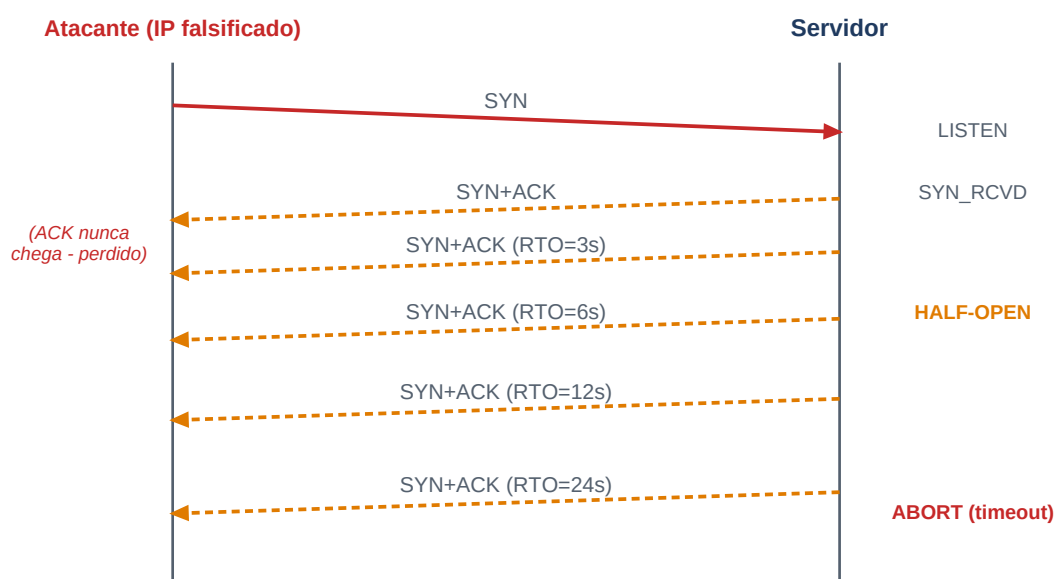


Figura 2 - SYN Flood: o servidor retransmite vários SYN-ACK por SYN (RTO com backoff exponencial) até abortar a conexão.

4. Arquitetura e Integração da Funcionalidade

A funcionalidade foi implementada de forma modular, utilizando os recursos nativos do Scapy. A captura roda em segundo plano (AsyncSniffer) e cada pacote percorre os módulos de interpretação, detecção e registro. Uma interface gráfica (GUI), construída com Tkinter, exibe os dados em tempo real.

Arquitetura modular

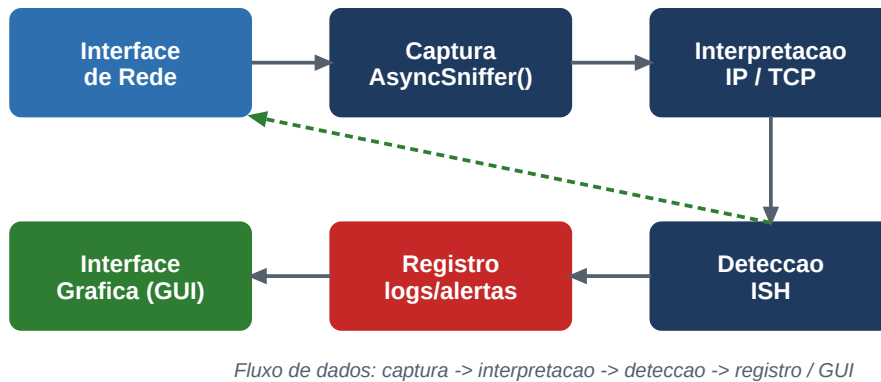


Figura 3 - Arquitetura modular do detector.

Componentes principais:

- **Captura de Pacotes** (sniffer/sniffer.py): obtém o tráfego com AsyncSniffer(), sem armazenar pacotes em disco;
- **Modulo de Interpretacao**: extrai os campos dos cabeçalhos IP e TCP e traduz as flags TCP para um formato legível;
- **Modulo de Deteccao** (detection/syn_flood_detector.py): faz a análise estatística por janela de tempo e calcula o índice ISH;
- **Modulo de Registro** (detection/registro.py): gera logs e alertas em arquivo usando o módulo logging;
- **Interface Grafica** (gui/): exibe a tabela de pacotes, estatísticas, gráfico de tendência e o status de segurança.

5. Interpretacao dos Cabecalhos de Protocolos (Evidencia)

Atendendo ao requisito obrigatorio de interpretacao de cabecalhos, o detector le e processa os seguintes campos dos protocolos capturados.

5.1. Protocolo IPv4

Campo	Finalidade
Source Address	Identificacao do endereco IP de origem
Destination Address	Identificacao do endereco IP de destino
Protocol	Verificacao do protocolo encapsulado (TCP)

5.2. Protocolo TCP

Campo	Finalidade
Source Port	Identificacao da porta de origem
Destination Port	Identificacao da porta de destino
Acknowledgment Number	Verificacao do estado da comunicacao
Flags	Identificacao dos estados SYN, ACK e SYN-ACK

As flags TCP interpretadas sao **SYN** (Synchronize), **ACK** (Acknowledgment) e a combinacao **SYN-ACK**, usadas para identificar tentativas de abertura de conexao e verificar se o handshake foi concluido.

5.3. Evidencia em codigo

O trecho abaixo (sniffer/sniffer.py) mostra a verificacao das camadas e a extracao efetiva dos campos dos cabecalhos IP e TCP de cada pacote capturado:

```
def analisar_pacote(self, pkt):
    if IP in pkt and TCP in pkt:
        # filtra apenas IPv4 + TCP
        origem = pkt[IP].src # IPv4 Source Address
        destino = pkt[IP].dst # IPv4 Destination Address
        source_port = pkt[TCP].sport # TCP Source Port
        destiny_port = pkt[TCP].dport # TCP Destination Port
        flag = self.traduzir_flags(pkt[TCP].flags) # Flags TCP
        pacote_info = {
            'flag': flag, 'origem': origem, 'destino': destino,
            'source_port': source_port, 'destiny_port': destiny_port,
            'time': pkt.time,
        }
        self.detector.analisar(pacote_info)
```

A traducao das flags converte o campo bruto do TCP em rotulos legiveis (ex.: "S" -> SYN; "SA" -> SYN + ACK), permitindo classificar cada pacote:

```
mapa = {'S': 'SYN', 'A': 'ACK', 'F': 'FIN', 'R': 'RST', 'P': 'PSH', 'U': 'URG'}
resultado = [mapa[l] for l in str(flags) if l in mapa]
return ' + '.join(resultado) # ex.: 'SYN + ACK'
```

6. Logica de Deteccao

O processamento de cada pacote segue o fluxo abaixo:

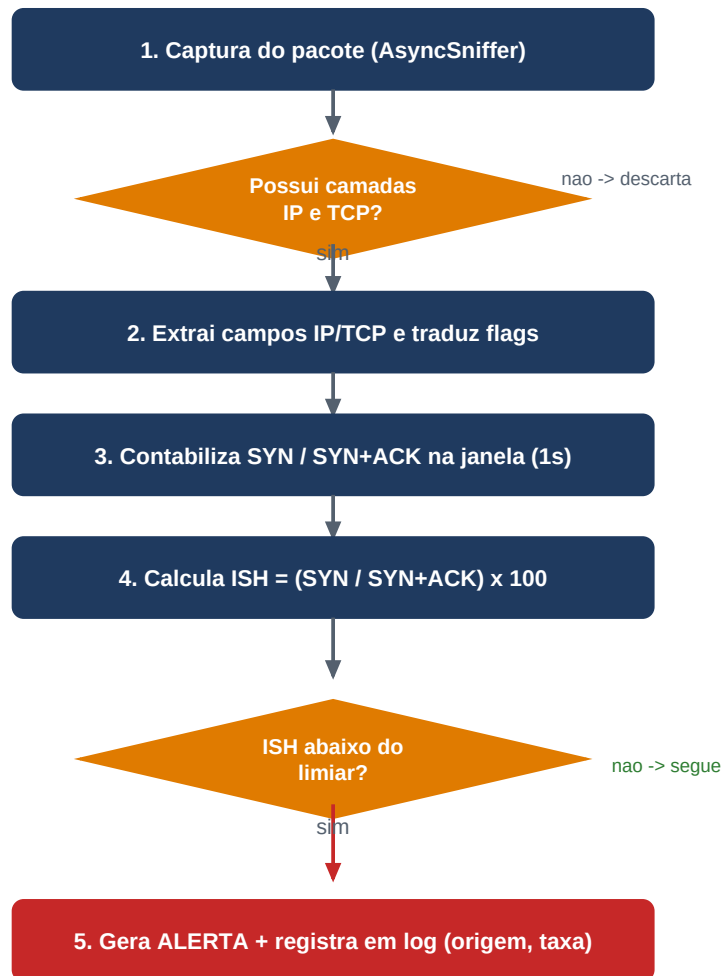


Figura 4 - Fluxo de deteccao do SYN Flood.

6.1. Calculo do indice (ISH)

A deteccao usa o indice **ISH** (Indice de Semiabertas), calculado dentro de uma janela de 1 segundo:

$$ISH = (\text{Total de SYNs recebidos} / \text{Total de SYN-ACKs enviados}) \times 100$$

Em trafego normal cada SYN gera aproximadamente um SYN-ACK, logo o ISH fica proximo de **100%**. Durante um ataque, o servidor retransmite varios SYN-ACK por SYN, o denominador cresce e o ISH **cai bem abaixo de 100%**. E essa queda que sinaliza o ataque. Trecho do detector (detection/syn_flood_detector.py):

```
syn = self.Syn_packages      # SYN recebidos na janela
ack = self.SynAck_packages   # SYN-ACK enviados na janela
if ack == 0:
    percentual = 100
else:
    percentual = (syn / ack) * 100    # ISH
```

6.2. Classificacao do status

O valor do ISH e convertido em um status de segurança:

Faixa de ISH	Status	Interpretacao
ISH > 85%	Normal	Trafego dentro do esperado
50% < ISH <= 85%	Situacao anormal	Possivel inicio de anomalia
ISH <= 50%	Possivel SYN Flood	Excesso de SYN-ACK por SYN

7. Registro de Eventos e Alertas

O Modulo de Registro utiliza a biblioteca padrao logging para gravar eventos no arquivo logs/syn_flood.log. Eventos normais sao registrados em nivel INFO e os alertas de ataque em nivel WARNING, contendo informacoes da origem do trafego suspeito (IPs mais frequentes), o valor do ISH e as contagens da janela.

Exemplo de linhas de log geradas:

```
2026-06-30 22:14:03 [INFO] Captura iniciada na interface padrao
2026-06-30 22:15:11 [WARNING] ALERTA SYN Flood: ISH=18.7% | SYN=420
                               SYN-ACK=2240 | origem suspeita=203.0.113.7
2026-06-30 22:16:02 [INFO] Situacao normalizada: ISH=98.3%
```

Observacao (conforme escopo): em ataques com IP falsificado, o endereco de origem tem valor limitado para a deteccao e serve principalmente para enriquecer os logs.

7.1. Estatisticas complementares

A interface e os logs disponibilizam ainda:

- **Media de SYNs** por segundo dentro do historico observado;
- **Maior pico de SYNs** registrado em uma janela;
- **IPs possiveis atacantes**: origens com maior numero de conexoes semiabertas (sem handshake concluido).

8. Procedimentos de Execucao

8.1. Pre-requisitos

- Python 3.12 ou superior;
- Biblioteca Scapy (pip install scapy);
- Biblioteca Matplotlib para o grafico (pip install matplotlib);
- No Windows, o driver de captura **Npcap** instalado;
- Privilegios de administrador (a captura de pacotes exige acesso a interface de rede).

8.2. Obtencao do codigo

```
git clone https://github.com/DennyIsBack/detector-syn-flood
cd detector-syn-flood
pip install -r requirements.txt
```

8.3. Execucao

Em um terminal aberto como administrador, na raiz do projeto:

```
python main.py
```

A janela principal ("Monitor TCP") abre exibindo: a tabela de pacotes capturados (flag, IP de origem e IP de destino), o painel de estatisticas (SYN, SYN-ACK, ISH e status), o botao "Grafico" (tendencia ao longo do tempo) e o botao "Enviar SYN" (gera um pacote SYN de teste).

9. Procedimentos de Validacao

A validacao verifica o comportamento do detector em dois cenarios: trafego normal e trafego sob ataque simulado em ambiente controlado.

9.1. Cenario 1 - Trafego normal

- Iniciar o programa e navegar normalmente na internet / executar pings;
- **Resultado esperado:** ISH proximo de 100%, status "Situacao Normal" (verde) e registro INFO no arquivo de log.

9.2. Cenario 2 - Ataque simulado

Gerar um grande volume de pacotes SYN (de varias origens, sem concluir o handshake) contra uma porta em escuta, em ambiente de laboratorio. Um gerador simples pode ser construido a partir de sniffer/sender.py:

```
from scapy.all import IP, TCP, send
from random import randint

for _ in range(2000):
    origem = '203.0.113.%d' % randint(1, 254) # IP falsificado
    pkt = IP(src=origem, dst='127.0.0.1') / TCP(dport=80,
        sport=randint(1024, 65535), flags='S')
    send(pkt, verbose=False)
```

Resultados esperados (evidencias de validacao):

- O ISH cai abruptamente para bem abaixo de 100%;
- O status muda para "Possivel SYN Flood detectado" (vermelho);
- E gravada uma linha WARNING em logs/syn_flood.log com o ISH, as contagens e o(s) IP(s) de origem suspeito(s);
- Media e pico de SYNs aumentam no painel de estatisticas;

- Ao cessar o ataque, o ISH retorna a ~100% e é registrada a normalização.

Aviso ético: o gerador de tráfego deve ser usado exclusivamente em ambiente próprio e controlado (ex.: loopback ou rede de laboratório isolada), para fins de teste e estudo.

10. Histórico de Desenvolvimento

O desenvolvimento foi versionado com Git e publicado no GitHub, mantendo um histórico de commits compatível com a evolução do projeto (estrutura inicial, correção da lógica de detecção, módulo de registro, estatísticas e ajustes de interface), em vez de um único commit final.

Repositório: <https://github.com/DennyIsBack/detector-syn-flood>

11. Documentação e Referências

- Scapy - <https://scapy.net/> e <https://github.com/secdev/scapy>
- [1] Wang, H.; Zhang, D.; Shin, K. G. *Detecting SYN Flooding Attacks*. IEEE INFOCOM 2002, pp. 1530-1539, 2002.
- [2] Divakaran, D. M.; Murthy, H. A.; Gonsalves, T. A. *Detection of SYN Flooding Attacks using Linear Prediction Analysis*. IIT Madras, 2006.
- [3] RFC 2988 - <https://datatracker.ietf.org/doc/html/rfc2988>
- [4] RFC 1122 - <https://datatracker.ietf.org/doc/html/rfc1122>